

רמזים למטלת הכיתה הרביעית

שבוע: 4 זמן משוער: פתח רק כשנתקעת

איך משתמשים בקובץ הזה

אין מתרגל בקורס, ולכן זה מה שמחליף מישהו שעובר בין השולחנות. לכל קושי שלוש רמות: **רמז** (כיוון) · **אסטרטגיה** (איך ניגשים) · **כמעט-פתרון** (הקוד כמעט שלם). פתח את הרמה הבאה רק אחרי שניסית באמת חמש דקות. מי שקופץ ישר לרמה 3 לומד להעתיק.

לפני הרמזים: כל קושי כאן הוקלד היום במחזור. הקוד שכתבת שם — `live-code/01-stack-to-row/end/styles.css` — הוא הרמז הראשון, והוא שלך.

הקושי	המחזור שבו הקלדת את זה	הקובץ
א · <code>display: flex</code> ולא זז כלום	מחזור 1, צעד 3 (הטעות המכוונת)	<code>live-code/01-stack-to-row/end/styles.css</code>
ב · התפריט	מחזור 5	<code>live-code/05-menu-closes-on-mobile/end/styles.css</code>
ג · המותג והתפריט	מחזור 1, צעדים 6-7	<code>live-code/01-stack-to-row/end/styles.css</code>
ד · ה- <code>hero</code>	מחזור 3, צעד 2 (עם שני פריטים)	<code>live-code/03-card-row-wraps/end/styles.css</code>
ה · הכרטיסים	מחזור 3, צעדים 1-2	<code>live-code/03-card-row-wraps/end/styles.css</code>
ו · הכפתורים בחבילות	מחזור 3, צעד 6 וה- <code>"אתה עושה"</code>	<code>live-code/03-card-row-wraps/end/styles.css</code>
ז · גלילה אופקית	מחזור 3, צעדים 3-5	<code>live-code/03-card-row-wraps/end/styles.css</code>
ח · התג	מחזור 2, צעדים 1-3	<code>live-code/02-badge-and-anchor/end/styles.css</code>

א. "שמתי `display: flex` ושום דבר לא זז"

רמז. `display: flex` משפיע על **הילדים הישירים** של האלמנט שכתבת עליו, ועל כלום אחר. שאל את עצמך: מי הילדים הישירים כאן, ובכמה מהם מדובר?

אסטרטגיה. פתח את כלי הפיתוח, בחר את האלמנט, ותסתכל בלשונית Elements. ליד אלמנט שהוא מכל `flex` יופיע תג קטן שכתוב עליו `flex` — **לחץ עליו**. הדפדפן יצייר לך על העמוד את הצירים, את הרווחים ואת המקום הפנוי. אם התג לא מופיע, ההצהרה לא תפסה, וזו בעיית בורר ולא בעיית פריסה.

שתי סיבות נפוצות שהצהרה לא תפסה: כתבת אותה על הילד במקום על ההורה, או שהבורר שלך לא מתאים לאף אלמנט (בדוק את הכתיב).

כמעט-פתרון.

```
/* not on the item – on whatever holds the items */
.card-row {
  display: flex;
}
```

ב. "התפריט לא נפתח / לא נסגר / תמיד פתוח"

רמז. `details` פותח ונסגר לבד, בלי שורת JavaScript אחת. אם הוא לא מתנהג – כנראה ה-CSS שלך מכריח אותו. ואם הוא **תמיד** פתוח מעל הסף, זו לא תקלה: זה בדיוק מה שביקשנו בסעיף 4.

אסטרטגיה. יש כאן שני מצבים שונים שקל לבלבל ביניהם:

1. **מתחת ל-48em** – הפקד מוצג, הקישורים מוסתרים עד שלוחצים. **אל תיגע בזה.** זו התנהגות ברירת המחדל של `details` והיא כבר עובדת.

2. **מעל 48em** – הפקד נעלם, והקישורים מוצגים **בלי קשר** למצב הפתיחה.

הטעות הכי נפוצה: לכתוב את הכלל של מצב 2 מחוץ לשאילתת המדיה. אז הקישורים מוצגים תמיד, גם בנייד, והפקד מיותר.

כמעט-פתרון.

```
@media (min-width: 48em) {
  .nav-toggle {
    display: none;
  }

  /* the two given lines are already here. do not delete them. */

  .nav-disclosure .nav-links,
  .nav-disclosure[open] .nav-links {
    /* the rule has to hold in both states, open and closed – hence both selectors */
  }
}
```

ג. "המותג והתפריט לא מתיישרים / לא מתרחקים זה מזה"

רמז. אלה שתי שאלות שונות. יישור אנכי הוא הציר המשני; ההרחקה היא הציר הראשי.

אסטרטגיה. שתי דרכים נכונות להרחקה, ואתה בוחר:

- `justify-content: space-between` על **המכל** – "סדר את הפריטים בקצוות". קורא טוב כשיש בדיוק שניים והסידור הוא הנקודה.
- `margin-inline-start: auto` על **הפריט האחרון** – "דחוף אותי לקצה". קורא טוב כשיש חמישה פריטים ואתה רוצה שרק אחד ייפרד מהשאר.

הרעיון מאחורי השנייה הוא זה: **מרווח אוטומטי בולע את כל המקום הפנוי**. בתוך flex זה הופך אותו מכלי מרכז לכלי מיקום.

כמעט-פתרון.

```
.site-nav {
  display: flex;
  align-items: center; /* the cross axis – this is what centres vertically */
  justify-content: space-between;
  gap: var(--space-4);
}
```

ד. "ה-hero לא מתקפל בנייד" או "האיור נמתח ומכווץ"

רמז. שתי בעיות נפרדות, ולשתיהן תשובה בת מילה אחת. הראשונה: `flex-wrap`. השנייה: `align-items`.

אסטרטגיה. לגלישה – מכל `flex` הוא `nowrap` כברירת מחדל, כלומר "תדחסו, אל תגלשו". תן לו רשות לגלוש, ותן לטקסט `flex-basis` שאומר מאיזה רוחב הוא מתחיל. כשאין מקום ל-`basis` ולאורך יחד, האיור יורד לשורה משלו.

לאורך – ברירת המחדל של `align-items` היא `stretch`, כלומר "מתחו את כל הפריטים לגובה השורה". על `img` זה נראה בדיוק כמו שזה נשמע. הפתרון הוא למרכז את הפריטים במקום למתוח אותם, וזה גם מה שאתה רוצה ויזואלית.

כמעט-פתרון.

```
.hero {
  display: flex;
  flex-wrap: wrap; /* without this it never wraps */
  align-items: center; /* without this the illustration stretches */
  gap: var(--space-6);
}

.hero-text {
  flex: 1 1 26rem; /* "I want 26rem. if there is not room, take a whole row" */
}
```

ה. "הכרטיסים לא נערמים בנייד" או "הם נערמים תמיד"

רמז. אלה בדיוק שני הצדדים של אותו מספר. ה-`flex-basis` שנתת לכרטיס הוא **הרוחב שהוא מבקש**, וממנו נגזר כמה מהם נכנסים לשורה.

אסטרטגיה. עשה את החשבון בקול. אם ה-`basis` הוא `17rem` ויש שני רווחים של `1.5rem`, שלושה כרטיסים דורשים בערך `54rem` – כלומר בערך 864 פיקסלים. מעל זה נכנסים שלושה; מתחת לזה אחד מהם גולש.

• נערמים תמיד? ה-`basis` גדול מדי (או שכתבת `width` במקום `flex-basis`, ואז אין ל-`flex` מה לחלק).

• לא נערמים לעולם? חסר `flex-wrap: wrap`, או שנתת `basis` באחוזים שתמיד מסתדר.

אל תוסיף שאילתת מדיה כאן. אם אתה מרגיש שאתה חייב – המספר לא נכון, לא הכלי.

כמעט-פתרון.

```
.card-row {
  display: flex;
  flex-wrap: wrap;
  gap: var(--space-5);
  padding: 0; /* a ul arrives with padding as well as bullets */
  margin: 0;
  list-style: none;
}

.card {
  flex: 1 1 17rem; /* this number IS the decision. change it and look. */
}
```

ו. "הכפתורים בחבילות לא מיושרים לתחתית"

רמז. החבילות כבר באותו גובה — זה קרה לבד. השאלה היא איפה בתוך החבילה יושב הכפתור, וזו שאלה על **הפריסה הפנימית של החבילה**, לא על השורה.

אסטרטגיה. שני שלבים, ובסדר הזה:

- הפוך כל חבילה בעצמה למכל flex, בכיוון עמודה. עכשיו הכותרת, המחיר, הרשימה והכפתור הם פריטים על ציר אנכי.
- תן לכפתור `margin-block-start: auto`. אותו רעיון בדיוק כמו בסעיף ג': **מרווח אוטומטי בולע את כל המקום הפנוי** — וכל המקום הפנוי נמצא מעליו.

זה עובד בלי קשר לכמה שורות יש ברשימה, וזו הנקודה.

כמעט-פתרון.

```
.plan {
  display: flex;
  flex-direction: column;
}

.plan .button {
  margin-block-start: auto;
}
```

ז. "יש לי פס גלילה אופקי ואני לא מוצא ממה"

רמז. כמעט תמיד זה פריט flex שמסרב להתכווץ. ה- `min-width` שלו הוא `auto` ולא `0`, והוא לא ייכנס מתחת לרוחב התוכן שלו.

אסטרטגיה. קודם כול, **מצא את האשם** במקום לנחש. פתח את הקונסול והדבק:

```
[...document.querySelectorAll('*')].filter(
  (el) => el.getBoundingClientRect().right > document.documentElement.clientWidth + 1,
);
```

מה שיחזור הוא רשימת האלמנטים שחורגים ימינה. האשם הוא בדרך כלל **הראשון ברשימה**, ולרוב הוא פריט flex שיש בו כותרת ארוכה, תמונה, או מילה שאי אפשר לשבור.

התיקון הוא על **הפריט**, לא על המכל, ולא על `flex-shrink` — אפשר להעלות אותו ל-10 ולא יקרה כלום, כי shrink מותר לכווץ רק עד המינימום.

כמעט-פתרון.

```
.hero-text {
  flex: 1 1 26rem;
  min-width: 0;          /* the line that lifts the floor */
}
```

ח. "התג נחת בפיתת המסך במקום בפיתת הכרטיס"

רמז. ה-CSS של התג נכון לגמרי. הבעיה היא במי שהוא נמדד ממנו, וזה לא הוא — זה ההורה.

אסטרטגיה. `position: absolute` הולך למעלה בעץ עד שהוא מוצא אב שה- `position` שלו אינו `static`, ומודד ממנו. אם אין אף אחד — הוא מודד מהעמוד. **הוא לא זורק שגיאה ולא כותב אזהרה**, ולכן זה מבלבל.

בדוק בקונסול מי באמת מעגן אותו:

```
document.querySelector('.card-badge').offsetParent;
```

אם מה שחזר אינו הכרטיס — מצאת את הבאג.

התיקון הוא על הכרטיס, והוא לא מזיז כלום. זו הסיבה שקל לשכוח אותו.

כמעט-פתרון.

```
.card {
  position: relative; /* "measure from me" – and that is all it does */
}

.card-badge {
  position: absolute;
  inset-block-start: -0.75rem;
  inset-inline-end: 1rem;
}
```

ט. "סיימתי את הליבה. מה עכשיו?" – אופציונלי

לפי הסדר הזה (הסעיפים 3–5 הם העמקה, לא דרישה):

- שכבת ההרחבה** — שורת החבילות, שלב 6 במטלה (סעיף 7 בקובץ ה-CSS). הכפתורים המיושרים הם הדבר הכי שימושי שתלמד השבוע אחרי `flex-wrap`, והוא יחזור בכל כרטיס שתבנה אי פעם.
- שכבת האתגר** — שלב 7 במטלה (סעיף 8 בקובץ ה-CSS). הכותרת הנעוצה והתג.

3. **Flexbox Froggy** — עשרים דקות, עשרים וארבעה שלבים. אחריו שמות התכונות מפסיקים לדרוש חיפוש.

4. **שאלה אמיתית לחשוב עליה:** מה יקרה לשורת הכרטיסים אם תוסיף כרטיס רביעי? ואם תוסיף חמישי? נסה — **בלי לשנות שורת CSS אחת**. אם התשובה מפתיעה אותך, חזור ל- `flex-basis`.

5. אם יש לך זמן ורצון — פתח את `demos/06-flex/playground/` ונסה לשחזר בו את שורת הניווט שבנית, בלי להסתכל בקוד שלך.

6. **שקופיות העומק** (בתג `Deeper · self-study` ב- `deck.pdf`) — עומק אופציונלי, לא נבחן: הטבלאות המלאות, וההדגמות שמאחורי המחזורים.